

check_cm_lan_routes_authenticated

check_cm_lan_routes_authenticated.sh

Revision: 2 **Last modified:** 2026-08-26T00:00:00Z **Review round:** 2 (independent Fable review, §11.4.209 → §11.4.134) **Gate:** CM-LAN-ROUTES-AUTHENTICATED **Authority:** operator decision BOB-102 (2026-08-26) · §11.4.135 · §11.4.201

Overview

On 2026-08-26 the operator decided BOB-102 as “**Keep 0.0.0.0 + auth guard**”. The tunnel stays LAN-reachable — no bind address moves to 127.0.0.1 — and that exposure was accepted **on the condition** that a permanent regression guard fails the build if any LAN-reachable route ever stops demanding authentication. This gate is that condition made mechanical.

It enumerates every route served by a listener declared LAN-bound **through the registration idioms it models**, resolves per route whether authentication genuinely reaches it, and refuses the build when such a route is neither auth-wired nor covered by a justified, checked-in exemption. Registration forms it does *not* model fail closed rather than passing silently — see Modelled scope.

Prerequisites

- bash, python3 (≥ 3.9), and PyYAML.
- The sibling analyzer engine scripts/pre_build/lan_route_auth_analyzer.py.
- The policy file config/lan_route_auth_policy.yaml.

No container, no running service, and no network access is required — the gate is pure static analysis.

Usage

```
./scripts/pre_build/  
    check_cm_lan_routes_authenticated.sh           # default  
scope  
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --  
list      # JSON inventory  
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --policy  
P --root R  
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --help
```

Exit	Meaning
0	PASS — every LAN-reachable route is auth-wired or justly exempted
1	FAIL — one or more findings, each printed with resolved evidence
2	ERROR — usage error, missing analyzer/policy, or no usable python3

Why it does not grep for “auth”

A gate that greps for a token *mentioning* auth asserts a **proxy signal**, not the real condition (§11.4.201). A comment, a variable name or a log string would satisfy it while the route stayed wide open. This gate resolves the real wiring from the authoritative source, per framework:

Service kind	A route counts as protected when...
fastapi	the module parses with ast and a Depends(<marker>) genuinely reaches the route — as a handler parameter default, as the decorator’s dependencies=[...], or via its APIRouter(dependencies=[...]).
gin	an auth marker is actually installed with .Use(...) on the owning engine or route group. Group prefixes are resolved so the reported path is the real one.
gomux	the specific http.NewServeMux() value the route was registered on is the one an auth marker

Service kind	A route counts as protected when...
	wraps in a return. Resolution is per mux variable , never per file.

Per-mux wrap resolution

A whole-file “does an auth wrap appear anywhere” test would mark a second, bare mux as protected just because a *different* mux in the same file is wrapped — a **false attribution of safety**, which is categorically worse than failing to see a route at all. This gate resolves the wrap for the exact mux variable a route was registered on, following one level of assignment indirection, and treats an unresolvable wrap as **unwrapped** (fail closed).

Every refusal prints the route, its `file:line`, how the verdict was reached, and whether the method is mutating — so a false positive is diagnosable in one step rather than by re-deriving the gate’s reasoning.

Modelled scope and fail-closed behaviour

This gate does **not** claim to see every route a framework can express. It claims to see the forms in the table above, and to **fail closed** on any it does not model:

- FastAPI `add_api_route / add_route / .mount / .websocket`
- gin `.Any / .Match / .Handle / .Static* / .NoRoute / .NoMethod`
- a multi-line or computed (non-literal) path expression in any framework
- registration on the global `http.DefaultServeMux`

Each is reported as an UNMODELLED REGISTRATION IDIOM and **fails the gate**. It is never silently skipped. A guard whose stated purpose is catching tomorrow’s drift must not answer “clean” about a construct it cannot see (§11.4.201(7)(c) — the path is part of the instrument). Note that the zero-routes FALSE-NUL check cannot catch this case, because the service still resolves its *other* routes and returns a confident, wrong count.

To ship a route in an unmodelled form, either re-express it in a modelled one or extend the analyzer to model it — and add a fixture so it stays guarded.

False positives are failures too

A refusal on a genuinely public route is a FAIL-bluff of the same severity as a missed route (§11.4.201(1)). Deliberately-public routes are therefore declarable — but **never silently**. Every exemption entry requires:

- **class:** — either public-by-design or known-gap
- **justification:** — a non-empty explanation

The gate refuses an entry missing either, so a blanket skip is impossible.

known-gap entries are **real holes carried honestly**, not endorsements. The gate prints their count loudly on every run. They are expected to shrink and never grow (§11.4.261).

The drift property (why the route set is derived, never listed)

The inventory is re-derived from source on every run. A hand-maintained list drifts silently, and silent drift is exactly what this guard exists to stop. The consequence is the useful one:

- A **new route** is in neither the auth-wired set nor the exemption list, so it FAILS until somebody consciously protects or justifies it.
- A route whose auth wiring is **removed** drops out of the auth-wired set and is not exempted, so it FAILS — the literal regression BOB-102 is conditioned on.
- A **stale exemption** for a route that no longer exists FAILS, so the list cannot rot into fiction.
- A **duplicate exemption** for the same (service, method, path) FAILS, so a known-gap cannot be silently masked by a later public-by-design entry and the honest-gap count cannot be skewed (§11.4.261).
- An **unmodelled registration idiom** FAILS, so a route added tomorrow in a form this engine cannot parse cannot slip through unseen.
- A declared LAN service resolving **zero routes** FAILS as BLIND rather than passing quietly, because a blind extractor and a route-free service return the identical quiet zero (§11.4.201(6) FALSE-NULL).

Edge cases

- **Loopback services.** A service declared `lan_bound: false` is out of scope and is not scanned. Firing on it would be a false-positive refusal.
- **Redundant exemptions.** An entry for a route that *is* auth-wired is tolerated (it is harmless) rather than failed, to avoid a false positive on a defensively-listed route.
- **method: ANY.** Matches any method for that path — used for `stdlib-mux` routes, which multiplex methods inside the handler.
- **Aliased dependencies.** A module-level `_g = Depends(require_api_token)` used as `_`: `None = _g` is recognised as genuine protection, including when imported from a sibling module. Refusing it would be a spurious build failure, and a gate that cries wolf gets bypassed.
- **Malformed policy.** Unparseable YAML exits 2 (ERROR) with the real cause, never 1 with a misattributed “routes do not demand authentication”.
- **Method-scoped keys.** Exemptions are keyed on (service, method, path), so a path exempted for GET is **not** exempted when it later gains a POST.

Honest boundary (§11.4.6)

This gate asserts **static route wiring**: that each LAN-reachable route is attached to an auth mechanism. It does **not** assert that the mechanism is armed at runtime. The merge-service token gate is env-conditional — with `BOBA_API_TOKEN` unset, `require_api_token` returns immediately and the route is open in practice. That is a boot-time invariant (§11.4.254), not a static one, and belongs to a separate runtime check. Claiming otherwise here would be the bluff this guard exists to prevent.

The operator has since decided to arm `BOBA_API_TOKEN` and back it with a boot-time invariant; that work is tracked as **BOB-197**. This gate is the static half of the pair and does not supersede it.

Likewise the gate proves coverage of the routes it can resolve; it does not prove the auth mechanisms themselves are correctly implemented.

Related

- Policy/exemption data: `config/lan_route_auth_policy.yaml`
- Analyzer engine: `scripts/pre_build/lan_route_auth_analyzer.py`
- Paired §1.1 meta-test: `tests/pre_build/test_check_cm_lan_routes_authenticated.sh`

- Sibling gates: `check_cm_healthcheck_covers_served_ports.sh`,
`check_cm_killpg_pgid_guard.sh`